

세종 당뇨·혈당관리 모바일 앱 소스코드

소스코드 (발체) — 세종 당뇨·혈당관리 모바일 앱

프로그램의 명칭	세종 당뇨·혈당관리 모바일 앱 (Sejong_APP)
저작물의 종류	컴퓨터프로그램저작물
주요 기능	연속혈당측정(CGM) 기기 연동 및 혈당 자료 수집 · 혈당 지표(TIR/TAR/TBR/CV/GMI) 산출과 도표 표시 · 식사·운동 기록과 혈당 연관 분석 · 회원 인증 및 개인 의료정보 관리 · PWA 설치형 모바일 화면
개발 언어 및 환경	Java 17 (Spring Framework / 전자정부표준프레임워크) · JSP · JavaScript · MyBatis SQL(MySQL) · HTML5/CSS3 · Apache Tomcat 9
전체 소스 분량	약 33,800줄 (서드파티 라이브러리 및 KISA 제공 암호모듈 제외)
본 문서의 범위	전체 소스 중 주요 기능 6개 부분 발체. AI 혈당상담 챗봇 모듈은 별도 저작물로 등록하므로 이 문서에서 제외하였다.

① 로그인 세션 검사·만료 처리 [src/main/java/egovframework/sejong/util/SessionCheckInterceptor.java]

```
39 /** 로그인 전(비인증) 접근이 허용되어야 하는 엔드포인트 목록. 로그인/회원가입/약관/자동로그인 흐름 전체. */
40 private static final Set<String> PUBLIC_URIS = new HashSet<>(Arrays.asList(
41     "/index.do",           // 진입점(세션 유무에 따라 login/main 으로 분기)
42     "/loginPage.do",       // 로그인 화면
43     "/sendSensApi.do",     // 인증문자 발송
44     "/loginCheck.do",      // 인증번호 확인(세션 생성)
45     "/autoLogin.do",       // 자동로그인(세션 생성)
46     "/registerUser.do",    // 회원가입(세션 생성)
47     "/updateUser.do",     // 가입 흐름 중 회원정보 갱신(세션 생성)
48     "/User.do",           // 가입 흐름 중 전화번호 존재 확인
49     "/testUser.do",       // 테스트/간편 로그인(세션 생성)
50     "/testUser2.do",
51     "/testUser3.do",      // 카카오 이메일 로그인(세션 생성)
52     "/callback.do",       // 구글 OAuth 콜백
53     "/getSignList.do",    // 로그인 화면 약관 조회
54     "/alldelete.do",      // 로그인 화면 회원탈퇴
55     "/sampleAjax.do"
56     (... 중략 ...)
57 );
58 // 공개 URL 은 세션 검사 없이 통과
59 if (PUBLIC_URIS.contains(path)) {
60     return true;
61 }
62
63 HttpSession session = request.getSession(false);
64 Object user = (session == null) ? null : session.getAttribute("user");
65 if (user != null) {
66     return true; // 로그인 상태 → 통과
67 }
68
69 // 세션 만료/미로그인
70 if (isAjax(request)) {
71     response.setStatus(HttpServletResponse.SC_UNAUTHORIZED); // 401
72     response.setContentType("application/json;charset=UTF-8");
73     response.getWriter().write(
74         "{\n\"IsSucceed\":false,\n\"sessionExpired\":true,\n\"
75         + \"\nMessage\":\n\"세션이 만료되었습니다. 다시 로그인해 주세요.\n\"}");
76     response.getWriter().flush();
77 } else {
78     response.sendRedirect(ctx + "/loginPage.do");
79 }
80 return false;
81 }
```

② 연속혈당(CGM) 자료 수집 — 외부 기기 연동 및 토큰 자동 갱신

[src/main/java/egovframework/sejong/blood/web/BloodController.java]

```
339 @RequestMapping(value = "/getBloodData.do", method = RequestMethod.GET)
340 @ResponseBody
341 public ResponseObject getBloodData(HttpSession session, @RequestParam String start, @RequestParam String end,
    @RequestParam String accessToken
342     , @RequestParam String goTokenUrl) {
    (... 중략 ...)
369 // [2026-07-11] 서버측 자동 갱신: access token 만료(401/403)면 refresh_token 으로 재발급 후 1회 재시도
370 // (sejong-web 방식 이식 - 클라이언트 refresh 루프/재로그인 없이 투명하게 갱신)
371 if (responseCode == 401 || responseCode == 403) {
372     String refTok = bloodService.refreshToken(user.getUserUuid());
373     String newAcc = (refTok != null && !refTok.isEmpty()) ? refreshAndSaveToken(user.getUserUuid(), refTok) :
null;
374     if (newAcc == null) { json.IsSucceed = false; json.Data = "REAUTH"; return json; } // refresh 토큰도 만료
→ 재연동 필요
375     connection = (URLConnection) new URL(tokenUrl + "?" + query).openConnection();
376     connection.setConnectTimeout(5000); connection.setReadTimeout(8000);
377     connection.setRequestMethod("GET");
378     connection.setRequestProperty("Authorization", "Bearer " + newAcc);
379     responseCode = connection.getResponseCode();
380     System.out.println("Response Code(refresh): " + responseCode);
381 }
382 InputStream cgmIn = (responseCode >= 200 && responseCode < 300) ? connection.getInputStream() : connection.ge
tErrorStream();
383 BufferedReader in = new BufferedReader(new InputStreamReader(cgmIn));
384 StringBuilder response = new StringBuilder();
385 String inputLine;
386 while ((inputLine = in.readLine()) != null) {
387     response.append(inputLine);
388 }
389 in.close();
390 if (responseCode < 200 || responseCode >= 300) { json.IsSucceed = false; return json; } // 갱신 후에도 실패
391 System.out.println("Blood Data Response: " + response.toString());
392 String jsonResponse = response.toString();
```

③ 혈당 통계 산출 — 고·저혈당 발생 시간대 상위 3구간 [src/main/resources/egovframework/sqlmap/mapper/Blood_SQL.xml]

```
98 WITH RECURSIVE hours AS (
99     SELECT 0 AS h
100     UNION ALL
101     SELECT h + 1 FROM hours WHERE h < 23
102 ),
103 win AS (
104     SELECT
105         CONCAT(LPAD(h,2,'0'), ':00~', LPAD(h+1,2,'0'), ':00') AS win, -- 예: 08:00~09:00
106         h AS sort,
107         SUM(CASE WHEN t.UPT_VALUE > 0 AND t.UPT_VALUE < 70 AND HOUR(t.CGM_DTM) = h THEN 1 ELSE 0 END) AS hypo_cn
t,
108         SUM(CASE WHEN
                                t.UPT_VALUE > 180 AND HOUR(t.CGM_DTM) = h THEN 1 ELSE 0 END) AS hyper_c
nt
109 FROM hours
110 LEFT JOIN T_BLDINF_TRAN t
111     ON t.USER_UUID = #{userId}
112     AND t.CGM_DTM >= CONCAT("#{start}", ' 00:00:00')
113     AND t.CGM_DTM < DATE_ADD("#{end}", INTERVAL 1 DAY)
114     AND t.UPT_VALUE > 0
115     AND HOUR(t.CGM_DTM) = h
116 GROUP BY h
117 )
118 SELECT
119     IFNULL((
120         SELECT GROUP_CONCAT(win ORDER BY sort ASC SEPARATOR ' , ')
121         FROM (
122             SELECT win, sort, hypo_cnt
```

③ 혈당 통계 산출 — 고·저혈당 발생 시간대 상위 3구간 (이어서)

[src/main/resources/egovframework/sqlmap/mapper/Blood_SQL.xml]

```
123      FROM win
124      WHERE hypo_cnt >= 1
125      ORDER BY hypo_cnt DESC, sort ASC
126      LIMIT 3
127      ) s1
```

④ 혈당 변화 화살표 — 직전 측정값 대비 연속 각도 표시 [src/main/webapp/WEB-INF/jsp/main/FAHR_00.jsp]

```
886  ① 「화살표 방향 수정」 (↗ 인데 옆 숫자는 -2 였다) ② 「직관적으로 · 사용자에게 설명해야 하는
887  변수는 배제」 두 지적을 함께 반영한 결과 -
888  · 근거는 **바로 옆 두 수치의 차이(point)** 하나뿐. 분당 변화율(mg/dL/분)·30분 추세처럼
889  화면에 없는 값은 쓰지 않는다. **보고 있는 숫자가 곧 화살표**라 설명이 필요 없다.
890  · 오르면 위(↗·↑) · 내리면 아래(↘·↓) · 그대로면 수평(→). 색은 숫자와 같은 _dirColor.
891  · 기울기는 단계 없이 이어진다 : 0=수평 · ±ARROW_FULL 이상=수직. 이 상수 하나로 조절.
892  ※하단 '현재 혈당 변화' 상태·설명문은 **설명서의 「지난 30분」 기준 그대로**(applyTrend30). */
893  const arrowEl = document.getElementById('bloodArrow');
894  if (arrowEl) {
895      const ARROW_FULL = 10; // 직전 5분에 ±10 mg/dL 이면 수직
896      let _r = _hasBoth ? (point / ARROW_FULL) : 0;
897      if (_r > 1) _r = 1; else if (_r < -1) _r = -1;
898      const _deg = -(_r * 90); // 0=수평(→) · -90=위(↑) · +90=아래(↓)
899      arrowEl.textContent = _hasBoth ? '→' : '='; // 계산 불가는 '=' (수평 화살표는 '안정적'으로 읽힌다)
900      arrowEl.style.display = 'inline-block'; // transform 이 먹으려면 필요(기본 inline 이면 무시된다)
901      arrowEl.style.transition = 'transform .35s ease';
902      arrowEl.style.transform = 'rotate(' + _deg.toFixed(1) + 'deg)';
903      arrowEl.style.color = _hasBoth ? _dirColor : '#6c757d';
904      arrowEl.title = _hasBoth
905          ? (point > 0 ? '5분 전보다 ' + point + ' 높아졌습니다'
906            : point < 0 ? '5분 전보다 ' + Math.abs(point) + ' 낮아졌습니다'
```

⑤ 무활동 1시간 자동 로그아웃 [src/main/webapp/WEB-INF/tiles/main/header.jsp]

```
48  (function(){
49      var IDLE_MIN = 60; // 무활동 허용 시간(분) - web.xml session-timeout 과 같게
50      var LIMIT = IDLE_MIN * 60 * 1000;
51      var timer = null, last = 0, done = false;
52
53      function ctx(){
54          try { return CommonUtil.getContextPath(); } catch(e){ return ''; }
55      }
56      function bye(){
57          if (done) return; done = true;
58          /* 자동로그인을 깨야 로그아웃이 유효하다 - 안 끄면 로그인 화면에서 곧바로 다시 들어온다
59          (login/main.jsp 의 logout() 주석과 같은 이유). 앱이 아니면 조용히 넘어간다. */
60          try { callAndroid("f102", { phone: (window._userPhone || ''), autoYn:false, saveYn:true }); } catch(e){}
61          alert(IDLE_MIN + '분 동안 사용하지 않아 로그아웃되었습니다.\n다시 로그인해 주세요.');
```

나간다 */

```
62          var go = function(){ location.href = ctx() + '/loginPage.do'; };
63          try { CommonUtil.callAjax(ctx() + '/logout.do', 'POST', '', go); setTimeout(go, 1500); } /* 응답이 없어도
64              catch(e){ go(); }
65          }
66          function reset(){
67              if (done) return;
68              var now = +new Date();
69              if (now - last < 30000) return; // 30초 안에 또 들어온 움직임은 흘린다(마우스·스크롤이 초당 수십 번 온
다)
70              last = now;
71              if (timer) clearTimeout(timer);
72              timer = setTimeout(bye, LIMIT);
73          }
74
75          /* 사람이 화면을 만졌다고 볼 수 있는 것들. capture 로 잡아 화면 어디서 눌러도 걸린다. */
76          ['click','keydown','touchstart','mousedown','mousemove','wheel','scroll','input'].forEach(function(ev){
```

⑤ 무활동 1시간 자동 로그아웃 (이어서) [src/main/webapp/WEB-INF/tiles/main/header.jsp]

```
77     document.addEventListener(ev, reset, true);
78   });
```

⑥ 식사등록 음식 검색 — 터치 기기 대응 목록 닫기 [src/main/webapp/WEB-INF/jsp/main/food/foodMain.jsp]

```
686 function outsideClose(e){
687   if (e.target === inputName) return;
688   if (listbox.contains(e.target)) return;
689   hideList();
690 }
691 document.addEventListener('pointerdown', outsideClose, true);
692 document.addEventListener('touchstart', outsideClose, true);
693 document.addEventListener('mousedown', outsideClose, true); // 마우스 쓰는 PC 화면
694 /* 화면을 굴리면 닫는다 - 목록이 칸에 붙어 있어 굴리면 자리가 어긋난다(휴대폰에서 흔한 동작) */
695 /* 굴리면 닫는다 - **연 직후 0.8초 안의 스크롤은 무시**(키보드가 올라오며 생기는 스크롤) */
696 /* ★[2026-08-21] **목록 자체를 굴리는 스크롤은 닫지 않는다** - capture 라 목록 안 스크롤에도
697   이 핸들러가 깨어나, 검색 결과를 굴리려는 순간 목록이 닫혔다(「검색 후 스크롤이 안 됩니다」).
698   페이지(다른 곳) 스크롤은 종전대로 닫는다 - 목록이 칸에서 떨어져 보이는 것을 막는 원래 목적. */
699 window.addEventListener('scroll', function(e){
700   var t = e.target;
701   if (t && t.nodeType === 1 && (t === listbox || listbox.contains(t))) return; // 목록 안 스크롤
702   if (Date.now() - openedAt < 800) return;
703   hideList();
704 }, true);
705 document.addEventListener('keydown', (e)=>{ if (e.key==='Escape') hideList(); });
```

※ 본 문서는 전체 소스코드 중 주요 부분을 발췌한 것이며, 각 코드 왼쪽의 번호는 해당 파일 내 실제 줄번호이다. 가독성을 위해 발췌 구간의 공통 들여쓰기만 제거하였고 코드 내용은 원본과 동일하다. 보안상 접속정보·인증키 등 설정값이 담긴 파일은 발췌 대상에서 제외하였다.